The Format

- Time: 60 Minutes (Strict).
- Team: 5 People.
- Goal: A working prototype (CLI or Basic GUI) that passes specific test cases.
- Concept: Each challenge focuses on a different core Python pillar.

Idea 1: The "Code Climate" CLI Tool

Focus: *File I/O, String Manipulation, and Dictionaries.*

The Challenge:
Your team is a "Developer Productivity" squad. Build a CLI tool that scans a given Python file (provided by the judges) and returns a "Health Score" based on specific rules.

The Requirements:

1. Line Counter: Count the total lines, blank lines, and lines of code (excluding comments).
2. Complexity Check: Count the number of `if`, `elif`, `else`, `for`, and `while` statements.
3. Comment Ratio: Calculate the percentage of lines that are comments.
4. Variable Check: Find all variables assigned with `=` and ensure they follow `snake_case` (warn if not).

Output: Print a JSON-like dictionary to the terminal with the stats.

Team Splitting Strategy:

- Person 1: The File Reader & Line Counter.
- Person 2: Regex/Logic for finding keywords (`if`, `for`).
- Person 3: Comment detection logic (handling `#` and docstrings).
- Person 4: Variable extraction and naming rule validator.
- Person 5: The "Integrator" – builds the main function, compiles the dict, and formats the output.

Idea 2: The "Memory Labyrinth" Game

Focus: *Object-Oriented Programming (OOP) and Inheritance.*

The Challenge:
Build a text-based maze game. The maze is a 5x5 grid. The player (`@`) must find the treasure (`T`) while avoiding traps (`X`).

The Requirements:

1. Base Entity: Create a base `Entity` class with `x` and `y` coordinates and a `symbol` attribute.
2. Inheritance: Create subclasses: `Player` (moves via WASD), `Trap` (kills the player if stepped on), and `Treasure` (ends the game).
3. Encapsulation: The `Player` should have a `health` attribute (starts at 3). Traps reduce health by 1.
4. Game Loop: A `while` loop that renders the grid every turn and takes input.

Team Splitting Strategy:

- Person 1: The `Entity` Base Class and `Treasure` class.
- Person 2: The `Player` class (movement logic and health).
- Person 3: The `Trap` class and collision detection logic.
- Person 4: The Grid Renderer (printing the board nicely).
- Person 5: The Game Manager (initializing objects, running the main loop, handling win/lose conditions).

Idea 3: The "List-Processor" Web Scraper (Mock)

Focus: *List Comprehensions, Lambda Functions, and* `map`/`filter`.

The Challenge:
You are given a large `.csv` file containing 1000 rows of fake user data (Name, Email, Age, City, Purchase_Amount). You cannot use Pandas. You must process it purely using Python lists.

The Requirements:

1. Filter: Use `filter()` and a lambda to find all users over 30 who spent more than $100.
2. Map: Use `map()` to create a new list of emails for these users.
3. Comprehension: Use a list comprehension to generate a list of "Name: Age" strings for users in "New York".
4. Reduce: Use `functools.reduce` to calculate the *total* purchase amount of the entire dataset.
5. Sorting: Sort the top 5 oldest users and print their names.

Team Splitting Strategy:

- Person 1: Load the CSV file and parse it into a list of dictionaries/tuples.
- Person 2: Implement the `filter` and `map` for the "Over 30" requirement.
- Person 3: Implement the list comprehension for "New York".
- Person 4: Implement the `reduce` for total sum and the sorting logic.
- Person 5: Integrate everything into a single script and handle print formatting for the final results.

Idea 4: The "Error-Proof" Calculator API

Focus: *Exceptions, Decorators, and Error Handling.*

The Challenge:
Build a simple arithmetic API (functions) that *never* crashes, no matter what input the user gives.

The Requirements:

1. Decorator: Create a `@safe_divide` decorator that catches `ZeroDivisionError` and returns "Infinity" instead of crashing.
2. Type Checking: Create a `@validate_inputs` decorator that checks if the arguments are integers/floats. If not, raises a custom `InvalidInputError` (which you must define) and catches it.
3. Logging: Every time a function is called, log the arguments and result to a file `log.txt`.
4. Functions: Create `add`, `subtract`, `multiply`, and `divide` (with the decorators applied).

Team Splitting Strategy:

- Person 1: Define the custom `InvalidInputError` exception class and the `validate_inputs` decorator.
- Person 2: Define the `safe_divide` decorator and the logging mechanism (writing to file).
- Person 3: Write the simple `add` and `subtract` functions and apply the decorators.
- Person 4: Write `multiply` and `divide` and apply the decorators.
- Person 5: Write a "Testing Harness" – a loop that asks the user for inputs and calls the functions, intentionally testing edge cases (strings, zero, etc.) to prove the code doesn't crash.

Idea 5: The "Task Scheduler" (Time Complexity Focus)

Focus: *Algorithms,* `datetime`*, and Big-O optimization.*

The Challenge:
You have a list of tasks: `tasks = [{"name": "A", "priority": 2, "deadline": "2026-06-20"}, ...]`. Build a tool to manage them.

The Requirements:

1. Parsing: Convert string dates to `datetime` objects.
2. Sorting: Sort the tasks by priority (1 is highest) and then by deadline (earliest first). *Constraint:* Must use a custom `key` function, not `itemgetter`.
3. Filtering: Find tasks due within the next 3 days.
4. Optimization (The Hack): The list has 100,000 items. Implement a search function to find a task by name. Must use a Dictionary (Hash Map) for O(1) lookup, not a loop.

Team Splitting Strategy:

- Person 1: Generate mock data (100k tasks) and handle the `datetime` parsing logic.
- Person 2: Implement the complex sorting algorithm with custom priority keys.
- Person 3: Implement the "Next 3 Days" filter using `timedelta`.
- Person 4: Build the Dictionary index (Name -> Task) for O(1) search.
- Person 5: Write the benchmark script – time how long the sort takes vs the dictionary lookup to prove the complexity difference.